

Name: Shivam Pokharkar

1) Status filter ignored + archived tasks down

⇒ "Done" filter still showed OPEN tasks, API status = DONE returned 49 Rows, Expected q = api returned archived tasks #20 #21

Where? — in Backend / SQL — Task Repository . Java

Root Cause :- Same query in db/queries/search_tasks.sql
AND runs before OR so the query meant
(archived AND title) OR (description AND status)
title match skipped the status check
description matches skipped the archived check

Fix :- Added brackets = archived AND (title OR description)
AND status. Added "id DESC" tiebreaker for a stable page order. Fixed both reference SQL files (COUNT + page query).

why :- Smallest change mapping matching the intended logic; Keeps H2 and Oracle copies consistent. Filtering in java instead would load hidden rows and keep total wrong.

2) Search showed result of older keystroke (race condition)

⇒ Typed "memory" one key at a time, the table showed result for "m"

Where? :- Frontend - hooks/useTask.js api.js

Root cause :- Every keystroke started a fetch with no cleanup, so the last response to arrive soon won. The backend made older shorter queries slower

Fix :- AbortController per request : the effect cleanup aborts the previous request aborted results are ignored.

Why :- Table addresses matches the input, even with network delays, and useless requests are cancelled. chosen over an ignore flag (does not cancel request) and debounce alone (does not guarantee order)

3) Stuck "loading..." on error; error never cleared

=> Simulated An API soo : "loading tasks..." forever after the API recovered, "Error soo" stayed until page reload

Where? :- frontend - hooks / useTasks.js

Root cause :- catch never set loading = false (Task table checks loading before error), and error was never reset

Fix :- setError (null) at the start of the request
setLoading (False) on error. loading starts as true

Why :- Three small state fixes make failure visible and let the app recover on its own. Not finally (): it also runs for aborted requests and would flicker

4) Pagination dead end ("No tasks found")

⇒ On page 5, Searched "cache" (3 matches): "No tasks found" and no pagination buttons

Where? :- Frontend - App.jsx

Root cause :- page was not reset when search or filter changed, so it asked for page 5 of a 1-page result.

Fix :- handleQueryChange / handleStatusChange set the value and page = 1

Why :- in the event handler React batches both updates → one render, one request. A useEffect would first send a wasted request for page 5.

5) Artificial Thread.sleep (Slow app, DOS risk)

⇒ Timing :- empty search ~ 1s, 10 character search 5 ms. Code slept $(10 - \text{query length}) \times 100 \text{ ms}$ labelled "Complexity estimation"

Where? :- Backend - TaskController.java

Root cause :- Deliberate sleep on the request thread with no real purpose

Fix :- Removed the block and the complexity value from the log.

Why :- Nothing used it, it slowly slowed every page load, caused bug 2 and each request blocked a Tomcat thread (max 200)

→ easy thread exhaustion. Empty search 1037 → 8 ms
Abuse protection belong in rate limiting, not sleeps

6) Bad input returned 500, no page limit

⇒ Edge case test : status = BOGUS, page = 0
 or -1, pageSize = -5, page = 2147483647
 → 500; page size = 100000 → all rows

Where? :- Backend - TestController.java

Root cause :- No validation: value of throw, sublist
 got a negative index and $(page - 1) \times page$
 pageSize overflowed int

Fix :- 400 for page < 1, pageSize outside 1-100
 or unknown status: offset calculated
 with log long

Why :- 4XX = client error, so 5XX alerts status
 meaningful: the limit stops bulk scraping
 error body does not echo input explicitly
 400 over. Silent clamping: manual
 check avoid a new dependency